

HNDIT 4012

Software Engineering

Complete Week-by-Week Academic Short Notes

Weeks 1 – 12

Introduction to SE • Software Process • Agile Development • Requirements Engineering • System Modeling
Software Architecture • Implementation • Testing • Maintenance • Project Management
Quality Management • Configuration Management

Week 1: Introduction to Software Engineering

This week introduces the discipline of software engineering, why it matters economically, the essential qualities good software must have, and the ethical responsibilities of software engineers.

1. What is Software Engineering?

- **Software engineering** is an engineering discipline concerned with all aspects of software production — from early specification through to maintaining the system after it goes into use.
- It applies appropriate theories, methods, and tools to solve problems, while respecting organisational and financial constraints.
- It covers more than coding: it also includes project management and the development of supporting tools and methods.

Software Engineering

An engineering discipline that applies systematic, disciplined and quantifiable approaches to the development, operation, and maintenance of software.

2. Why Software Engineering Matters

- Modern economies depend heavily on software — more systems are software-controlled every year.
- Software expenditure represents a significant share of GNP in developed countries.
- **Software costs** more to maintain than to develop; for long-lived systems maintenance costs can be several times the original development cost.
- Roughly 60% of software costs are development costs and 40% are testing costs; for custom software, evolution (maintenance) costs often exceed development costs.

3. Software Products

- **Generic products** — stand-alone systems marketed and sold to any customer (e.g. graphics software, project-management tools, CAD software).
- **Customised (bespoke) products** — software commissioned by a specific customer for their own needs (e.g. embedded control systems, air-traffic control software).

4. Essential Attributes of Good Software

- **Maintainability** — must evolve easily to meet changing customer needs.
- **Dependability and security** — reliable, safe, and resistant to malicious use; should not cause physical or economic harm.
- **Efficiency** — should not waste resources such as memory or processor cycles (responsiveness, processing time, memory use).
- **Acceptability** — must be understandable, usable, and compatible with the systems users already rely on.



Figure 1.1 — The four essential attributes every well-engineered software product should have.

5. General Issues Affecting Most Software

- **Heterogeneity** — systems increasingly run as distributed systems across many device and network types.
- **Business and social change** — rapid change forces software to evolve quickly and new software to be developed fast.
- **Security and trust** — because software is woven into everyday life, users must be able to trust it.

Application types

- **Stand-alone applications**
 - These are application systems that run on a local computer, such as a PC. They include all necessary functionality and do not need to be connected to a network.
- **Interactive transaction-based applications**
 - Applications that execute on a remote computer and are accessed by users from their own PCs or terminals. These include web applications such as e-commerce applications.
- **Embedded control systems**
 - These are software control systems that control and manage hardware devices. Numerically, there are probably more embedded systems than any other type of system.
- **Batch processing systems**
 - These are business systems that are designed to process data in large batches. They process large numbers of individual inputs to create corresponding outputs.
- **Entertainment systems**
 - These are systems that are primarily for personal use and which are intended to entertain the user.
- **Systems for modelling and simulation**
 - These are systems that are developed by scientists and engineers to model physical processes or situations, which include many, separate, interacting objects.

6. Software Engineering and the Web

- The web is now a platform for running applications; organisations increasingly build web-based rather than local systems.
- **Web services** let application functionality be accessed remotely over the web; **cloud computing** runs applications remotely on shared infrastructure.
- Reuse is the dominant approach for building web systems, which are usually developed and delivered incrementally rather than fully specified in advance.

7. Software Engineering Ethics

- Software engineering carries responsibilities beyond technical skill — engineers must act honestly and ethically to be respected as professionals.
- **Confidentiality** — respect the confidentiality of employers/clients, even without a signed agreement.
- **Competence** — do not misrepresent your skill level or accept work beyond your competence.
- **Intellectual property rights** — respect patents, copyright, and other IP law.
- **Computer misuse** — never use technical skills to misuse others' computers, from trivial misuse to serious acts like spreading viruses.
- The **ACM/IEEE Code of Ethics** sets out 8 principles: Public interest, Client and Employer, Product, Judgment, Management, Profession, Colleagues, and Self (lifelong learning).

Issues of professional responsibility

Confidentiality

- Engineers should normally respect the confidentiality of their employers or clients irrespective of whether or not a formal confidentiality agreement has been signed.

Competence

- Engineers should not misrepresent their level of competence. They should not knowingly accept work which is outwith their competence.

Intellectual property rights

- Engineers should be aware of local laws governing the use of intellectual property such as patents, copyright, etc. They should be careful to ensure that the intellectual property of employers and clients is protected.

Computer misuse

- Software engineers should not use their technical skills to misuse other people's computers. Computer misuse ranges from relatively trivial (game playing on an employer's machine, say) to extremely serious (dissemination of viruses).

Example — Case study — the insulin pump

- ▶ An embedded safety-critical system that reads a blood-sugar sensor and calculates the correct insulin dose.
- ▶ Low blood sugar can cause coma or death; high blood sugar causes long-term organ damage — so reliability is critical.
- ▶ Illustrates why software engineering discipline (not ad-hoc coding) is essential for safety-critical products.

Quick Revision Points

- ✓ Software engineering = engineering discipline covering the whole software lifecycle, not just coding.
- ✓ Good software must be maintainable, dependable & secure, efficient, and acceptable.
- ✓ Products are either generic (off-the-shelf) or customised (bespoke).
- ✓ Ethics and professional codes of conduct (e.g. ACM/IEEE) guide engineers' responsibilities.

Week 2: Software Process

A software process is the structured set of activities required to build a software system. This week covers the four fundamental process activities and the classic generic process models: waterfall, incremental development, and reuse-oriented engineering.

1. The Software Process

Software Process

A structured set of activities required to develop a software system. Different processes exist, but all involve specification, design & implementation, validation, and evolution.

- **Specification** — defining what the system should do.
- **Design and implementation** — defining the system's organisation and implementing it.
- **Validation** — checking the system does what the customer wants.
- **Evolution** — changing the system as customer needs change.

2. Plan-driven vs Agile Processes

- **Plan-driven processes** — all activities are planned in advance; progress is measured against the plan.
- **Agile processes** — planning is incremental, making it easier to adapt to changing requirements.
- Most real-world processes blend elements of both approaches.

3. Software Specification (Requirements Engineering)

- **Feasibility study** — is the system technically and financially feasible?
- **Requirements elicitation and analysis** — what do stakeholders actually need?
- **Requirements specification** — defining requirements in detail (produces the SRS).
- **Requirements validation** — checking that the requirements are correct and complete.

4. Software Design and Implementation

- Converts the specification into an executable system; design and implementation are closely related and often interleaved.
- **Architectural design** — overall system structure, principal components/sub-systems, and their relationships.
- **Interface design** — defines the interfaces between components.
- **Component design** — how each component will operate internally.
- **Database design** — the system's data structures and how they map to a database.

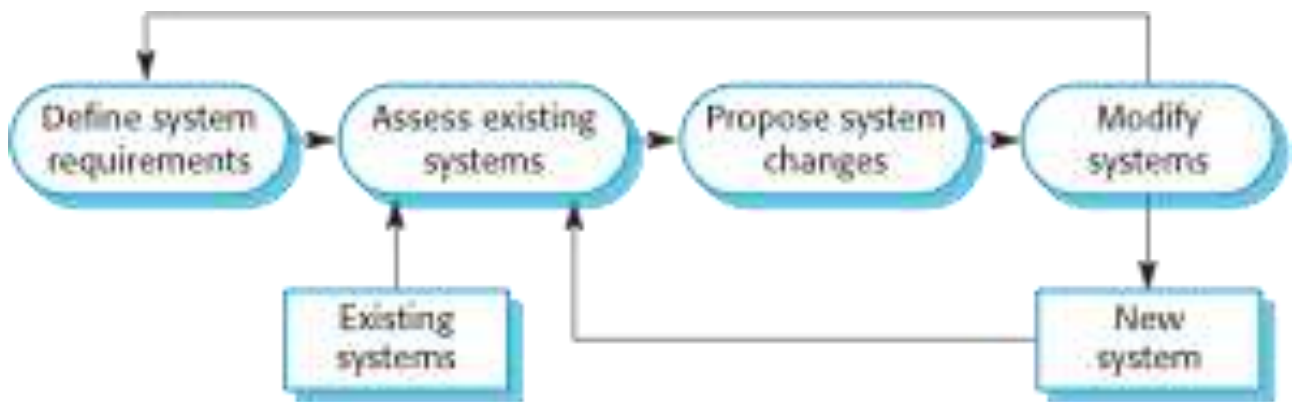
5. Software Validation (Testing Stages)

- **Development / component testing** — individual functions, objects or components tested independently.
- **System testing** — testing the system as a whole; emergent properties matter here.
- **Acceptance testing** — testing with real customer data to confirm the system meets customer needs.



6. Software Evolution

- Software is inherently flexible; as business needs change, systems must evolve to remain useful.
- The old line between “development” and “maintenance” is increasingly blurred, since very few systems today are completely new.



7. Software Process Models

(a) The Waterfall Model

- A **plan-driven** model with separate, sequential phases: Requirements analysis & definition → System & software design → Implementation & unit testing → Integration & system testing → Operation & maintenance.

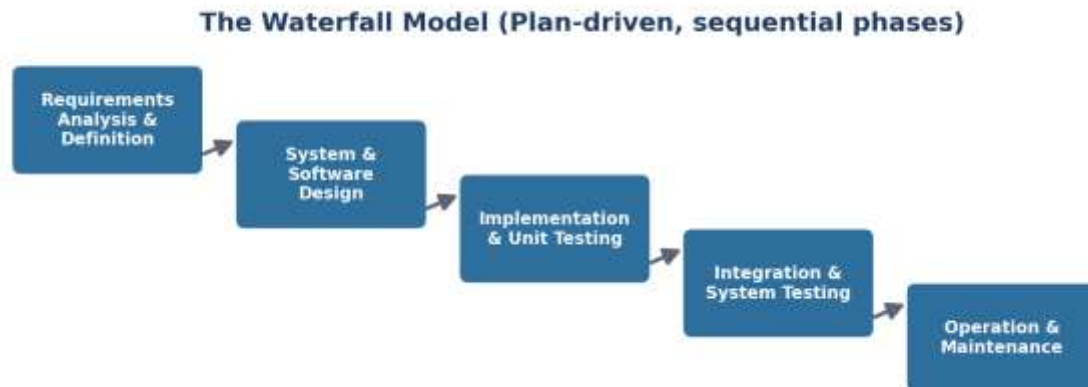
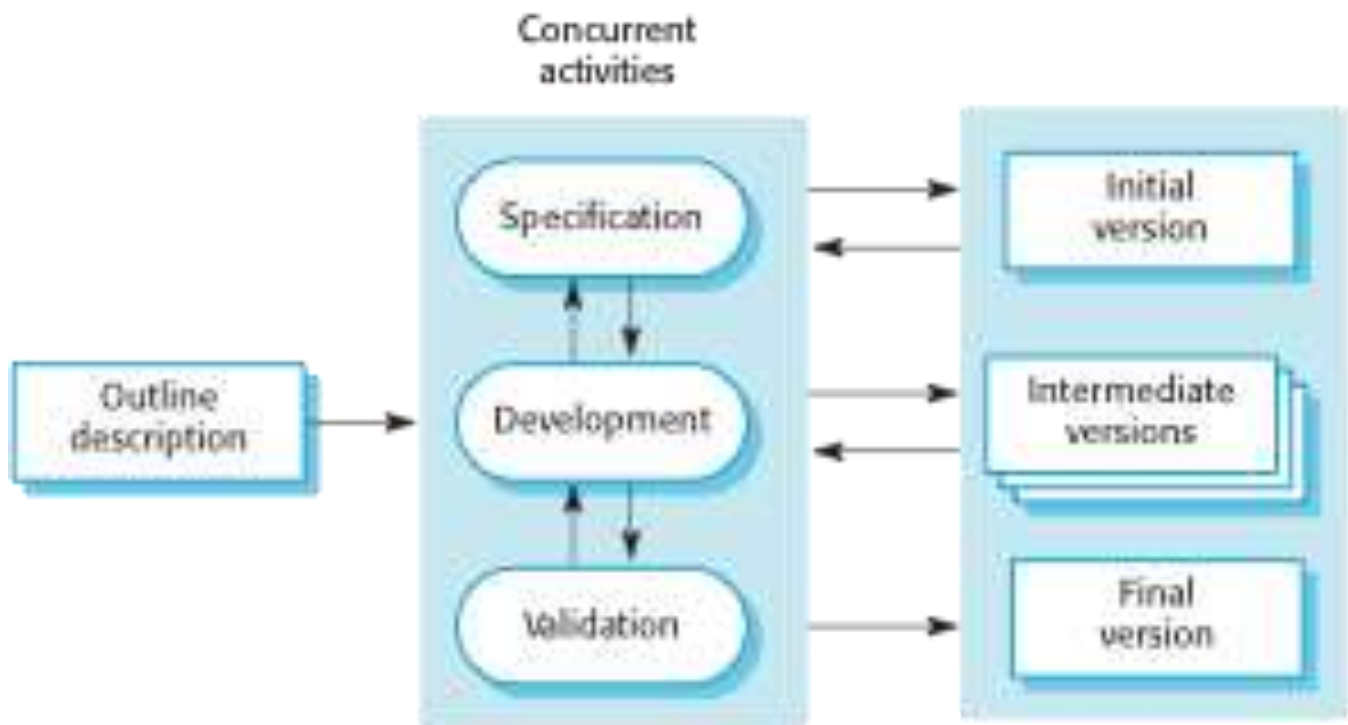


Figure 2.1 — The Waterfall Model: distinct, sequential phases.

- **Drawback:** hard to accommodate change once underway; rigid staging makes it unsuitable unless requirements are well understood and stable — few business systems fit this.

(b) Incremental Development (Evolutionary)

- Specification, development, and validation are interleaved rather than separated; may be plan-driven or agile.
- The system is delivered in **increments**; each increment adds part of the required functionality, prioritised by the user.
- Once an increment starts, its requirements are frozen — later increments can still evolve.
- **Advantages:** earlier delivery of value, early increments act as a prototype for eliciting later requirements, lower risk of total project failure.
- **Problems:** hard to identify shared/common facilities needed by all increments before they are all specified.
- Two sub-types: **Exploratory development** (e.g. Boehm's Spiral Model) and **Throw-away prototyping** (e.g. rapid prototyping).



(c) Boehm's Spiral Model

- Process is represented as a spiral, not a fixed sequence — each loop is a phase, chosen depending on what is needed.
- **Risks are explicitly assessed and resolved** throughout the process (a risk-driven approach).
- Each loop has four sectors: **Objective setting, Risk assessment & reduction, Development & validation, Planning.**
- Influential conceptually, but rarely used exactly as published in real projects.

(d) Software Prototyping

- A **prototype** is an initial version of a system used to demonstrate concepts and explore design options — useful in requirements elicitation, UI design, and back-to-back testing.
- **Benefits:** improved usability, closer match to real user needs, better design quality and maintainability, reduced overall development effort.
- **Throw-away prototypes** are normally discarded — they are usually undocumented, structurally degraded by rapid change, and unlikely to meet organisational quality standards.

(e) Reuse-Oriented Software Engineering

- Systems are assembled from existing components or **COTS** (Commercial-Off-The-Shelf) systems rather than built from scratch.
- Process stages: **Component analysis → Requirements modification → System design with reuse → Development and integration.**
- Component types include: web services, object collections built for a framework (.NET, J2EE), and stand-alone COTS systems.

Example — Choosing a process model

- ▶ A banking core system with stable, well-defined regulations → Waterfall may suit (requirements unlikely to change).
- ▶ A start-up building a new mobile app with evolving user feedback → Incremental / Agile development suits better.
- ▶ A company building a CRM using many existing SaaS APIs → Reuse-oriented engineering is a natural fit.

Quick Revision Points

- ✓ Every software process involves specification, design/implementation, validation, and evolution.
- ✓ Waterfall = sequential and plan-driven; good only when requirements are stable.
- ✓ Incremental development delivers the system in prioritised, working pieces.
- ✓ The Spiral Model adds explicit, repeated risk assessment to iterative development.
- ✓ Reuse-oriented engineering builds systems mainly from existing components/COTS.

Week 3: Agile Software Development

Agile development is a flexible, adaptive approach for building software when requirements are not fully known upfront. This week compares Agile to traditional plan-driven models and introduces Scrum, Extreme Programming (XP), ASD, and DSDM.

1. Why Agile?

- **Problems with traditional (plan-driven) models:** requirements assumed fixed, development follows a strict sequence, and changes introduced late are costly.
- In modern development, requirements change frequently, markets evolve quickly, and customer expectations keep rising.

Agile Software Development

A flexible and adaptive approach to building software in environments where requirements are not fully known at the start, emphasising iteration, incremental delivery, and continuous feedback.

2. Key Features of Agile

- **Iterative development** — build the software in repeated cycles.
- **Incremental delivery** — release usable pieces of functionality regularly.
- **Continuous feedback** — from customers throughout development, not just at the end.
- **Strong collaboration** — close, ongoing teamwork between developers and customers.
- Agile still performs the basic SE activities (communication, planning, modelling, construction, deployment) but keeps them lightweight and iterative.

3. Advantages and Limitations

- **Advantages:** faster delivery of usable software, improved customer satisfaction, better handling of change, continuous improvement via feedback.
- **Limitations:** needs active customer involvement, less emphasis on documentation, difficult to scale to very large systems, requires experienced/collaborative teams.

4. Scrum

- One of the most widely used Agile methods; organises work into short, fixed cycles called **sprints** (usually 2–4 weeks).
- **Product backlog** — the prioritised list of desired features.
- **Sprint planning** → **sprint execution** → **product increment delivery** — each sprint ends with a working increment.
- **Roles:** **Product Owner** (represents the customer), **Scrum Master** (facilitates the process), **Development Team** (builds the product).
- During a sprint, the scope is fixed — no new changes are introduced until the sprint ends.

Scrum Process Flow

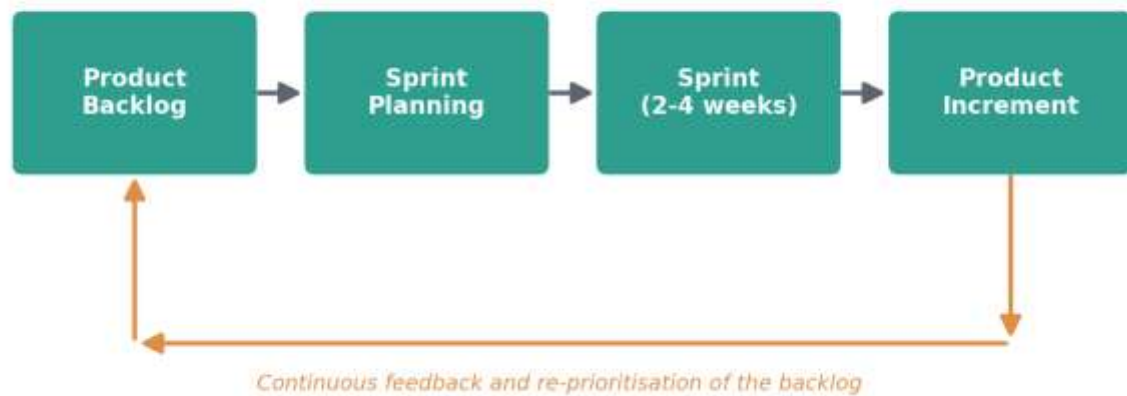


Figure 3.1 — The Scrum process flow: backlog, planning, sprint, and increment, with continuous feedback.

5. Extreme Programming (XP)

- Emphasises high-quality code, continuous feedback, and close customer involvement; focuses on **engineering practice** more than management process.
- Called “extreme” because it takes good practices and applies them intensively: **Testing** → **test all the time**; **Code review** → **pair programming**; **Design** → **keep it simple always**.
- **Advantages**: high-quality software, early error detection, strong teamwork, continuous customer feedback.
- **Limitations**: needs experienced developers and strong customer involvement; not well suited to large distributed teams; less documentation.

6. Other Agile Methods

- **Adaptive Software Development (ASD)** — three phases: **Speculation** (planning), **Collaboration** (development), **Learning** (feedback and improvement); focuses on teamwork and continuous learning.
- **Dynamic Systems Development Method (DSDM)** — focuses on delivering systems quickly using incremental prototyping within strict time constraints, ensuring useful software ships even under tight deadlines.

Example — Scrum in practice

- ▶ A team builds a food-delivery app. The product backlog lists features like 'user login', 'order tracking', and 'payment integration'.
- ▶ In Sprint 1 (2 weeks), the team commits to building 'user login' and 'restaurant browsing' only.
- ▶ At the sprint review, a working increment is demonstrated to the Product Owner, who reprioritises the backlog for Sprint 2 based on feedback.

Quick Revision Points

- ✓ Agile favours iteration, incremental delivery, and continuous customer feedback over rigid up-front planning.
- ✓ Scrum organises work into fixed-length sprints with a backlog, defined roles, and a working increment each sprint.
- ✓ XP emphasises intense application of good engineering practices: continuous testing, pair programming, simple design.
- ✓ ASD and DSDM are further Agile frameworks, each with their own emphasis (learning cycles / rapid time-boxed delivery).

Week 4: Requirements Engineering (RE)

Requirements engineering is the foundation of software development: discovering, analysing, documenting, and validating what a system must do and the constraints it must satisfy.

1. What is a Requirement?

Requirement

A description of a system service or a constraint on the system's operation, reflecting the genuine needs of the customer.

Requirements Engineering (RE)

The process of discovering, analysing, documenting, and validating requirements for a software system.

- **Importance of RE:** it is the foundation of software development, reduces the risk of project failure, improves communication between stakeholders, and saves cost and time overall.

2. Types of Requirements

- **User requirements** — high-level, natural-language descriptions written for customers and end-users.
- **System requirements** — detailed, technical descriptions used by developers that precisely define system behaviour.
- **Functional requirements** — describe *what the system should do* (e.g. “the system shall allow user login”); examples include user authentication, data processing, and report generation.
- **Non-functional requirements** — constraints that define the *quality* of the system, not specific behaviours.
- Non-functional requirements are grouped as: **Product** (performance, reliability), **Organisational** (standards, policies), and **External** (legal, regulatory).

Example — Functional vs non-functional requirements

- ▶ Functional: “The system shall allow a registered user to log in with a username and password.”
- ▶ Non-functional (performance): “The system shall respond to a login request in under 2 seconds.”
- ▶ Non-functional (security): “Only authorised users shall be able to access account data.”
- ▶ Non-functional (usability): “A new user should be able to complete registration without training.”

3. The Requirements Engineering Process

- Four core activities, carried out iteratively: **Elicitation** → **Specification** → **Validation** → **Management**.

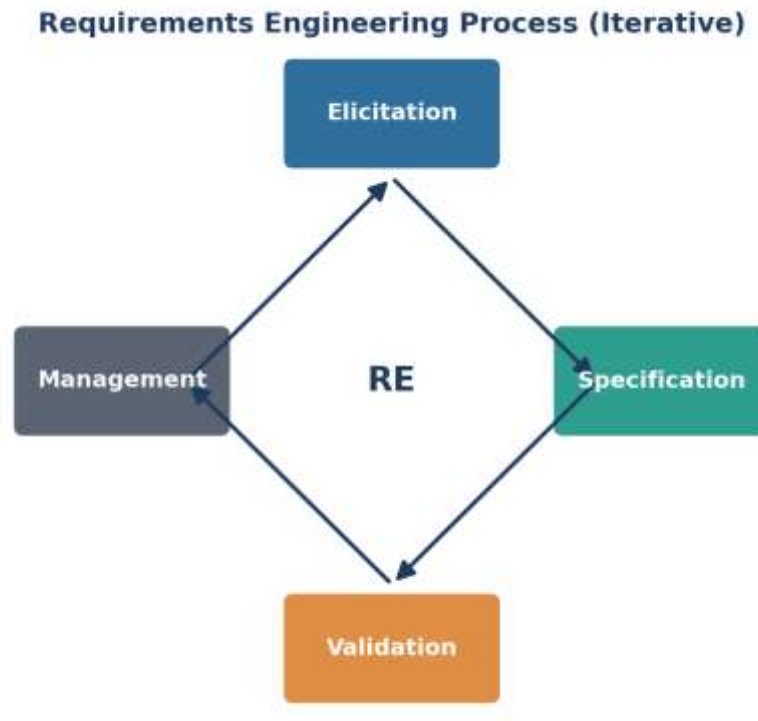


Figure 4.1 — The Requirements Engineering process is iterative, not a single one-way sequence.

(a) Requirements Elicitation

- Gathering requirements from stakeholders and understanding their real needs.
- **Challenges:** stakeholders may be unclear about their needs, requirements may conflict, communication gaps exist, and the environment keeps changing.
- **Activities:** discovery, classification, negotiation, and documentation.
- **Common techniques:** interviews, questionnaires/surveys, workshops and brainstorming, use cases and user stories, prototyping, observation, and document analysis.

(b) Requirements Specification

- Converting elicited requirements into a formal document — the **Software Requirements Specification (SRS)**.
- **Forms of specification:** natural language, structured language, graphical models (e.g. UML use cases), and mathematical models.
- **Good specification practice:** use a standard format, use “shall” for mandatory requirements, avoid ambiguity, and keep the wording simple.

(c) Requirements Validation

- Checking that requirements are correct and that the system, once built, will meet real user needs.
- **Checks performed:** validity, consistency, completeness, realism, and verifiability.
- **Techniques:** reviews, prototyping, and test-case generation.

(d) Requirements Management

- Managing and tracking requirement changes over the life of the project.
- **Change management process:** problem analysis → change analysis & costing → implementation.

Quick Revision Points

- ✓ A requirement describes a service or constraint that reflects genuine customer needs.
- ✓ Functional requirements define what the system does; non-functional requirements define its qualities/constraints.
- ✓ RE is iterative: Elicitation, Specification, Validation, and Management feed back into each other.
- ✓ Good requirements should be clear, testable (verifiable), and unambiguous.

Week 5: System Modeling (UML)

System modeling develops abstract representations of a system to simplify and communicate its design. This week covers the four modeling perspectives and the main UML diagram types used for each.

1. What is System Modeling?

System Modeling

The process of developing abstract models of a system, each representing a different perspective or view of that system.

- Represents a system visually, simplifies complex systems, and uses diagrams and graphical notation — most commonly **UML** (Unified Modeling Language).

2. UML — Unified Modeling Language

- A standard modeling language used across software engineering, supporting several diagram types: **Use Case, Sequence, Activity, Class, and State diagrams**.

3. Four Modeling Perspectives

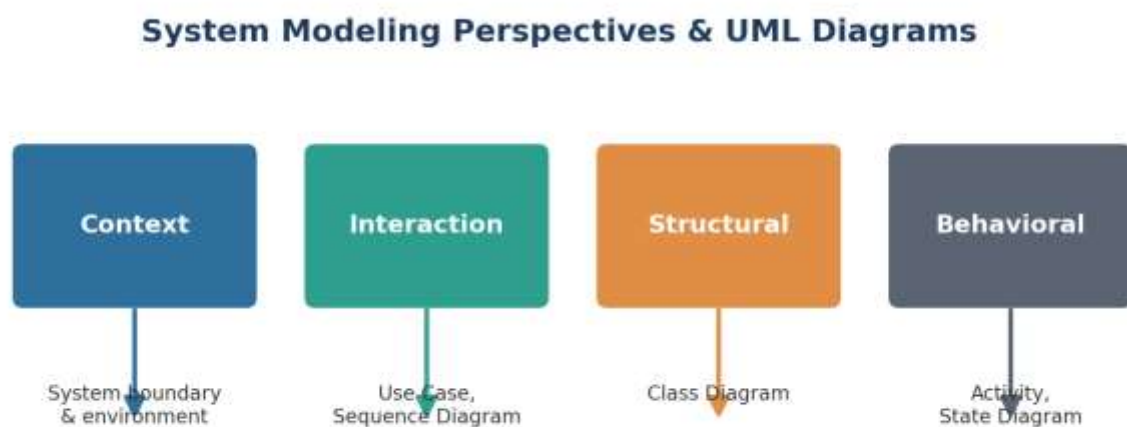


Figure 5.1 — The four system modeling perspectives and their associated UML diagrams.

(a) Context Models

- Show the **system boundary**, external systems, and the environment the system operates in.
- Example systems: an ATM system; an online shopping system, showing what lies inside vs outside the system boundary.

(b) Interaction Models

- Describe user interaction, system-to-system interaction, and component interaction.
- **Use Case Diagram** — shows actors, system functions, and how users interact with the system.
- **Sequence Diagram** — shows object interactions, the flow of messages, and the order of operations over time.

(c) Structural Models

- Show the organisation of the system — its classes and their relationships.
- **Class Diagram** — the main structural diagram; each class shows a **name**, **attributes**, and **methods**, connected via relationships such as **association**, **inheritance**, and **aggregation**.

(d) Behavioral Models

- Describe the system's dynamic behaviour: how it responds to events and processes data.
- **Activity Diagram** — shows workflow, process steps, and decision points.
- **State Diagram** — shows the system's possible states and the events that trigger transitions between them.
- **Data Flow Diagrams (data-driven modeling)** — show input data, processing steps, and output data.

Example — Modeling an online shopping system

- ▶ Context model: shows the online store bounded by external systems — payment gateway, courier service, and the customer's browser.
- ▶ Use case diagram: actor 'Customer' performs use cases 'Browse Products', 'Add to Cart', 'Checkout'.
- ▶ Class diagram: classes 'Customer', 'Order', and 'Product' linked by association (Customer places Order; Order contains Products).
- ▶ State diagram: an Order moves through states Placed → Paid → Shipped → Delivered.

4. Advantages of System Modeling

- Better communication among stakeholders, easier understanding of complex systems, improved documentation, reduced development errors, and easier maintenance.

Quick Revision Points

- ✓ System modeling builds abstract, visual representations of a system, usually using UML.
- ✓ Four perspectives: Context (boundary), Interaction (use case/sequence), Structural (class), Behavioral (activity/state).
- ✓ Class diagrams capture structure (attributes, methods, relationships); sequence diagrams capture the order of messages.
- ✓ Good models improve communication, documentation, and reduce downstream errors.

Week 6: Software Architecture

Software architecture defines the high-level structure of a system — its components, their relationships, and how they interact. This week surveys the major architectural patterns.

1. What is Software Architecture?

Software Architecture

The high-level structure or organisation of a software system, defining the components, relationships, and interactions that make up the system.

- **Advantages of explicit architecture:** supports stakeholder communication, enables system analysis, and supports large-scale reuse.

2. Architectural Patterns

An architectural pattern is a stylised, abstract description of good practice that has been tried and tested across many systems and environments.

(a) Model-View-Controller (MVC)

- Separates a system into **Model** (data/logic), **View** (presentation), and **Controller** (handles input and coordinates Model/View).
- Basis of interaction management in many web-based systems.

(b) Layered Architecture

- Organises the system into layers, each providing services to the layer above it.
- Supports **incremental development** — once a layer is built, some of its services may already be usable — and makes the architecture changeable and portable.

Generic Layered Architecture

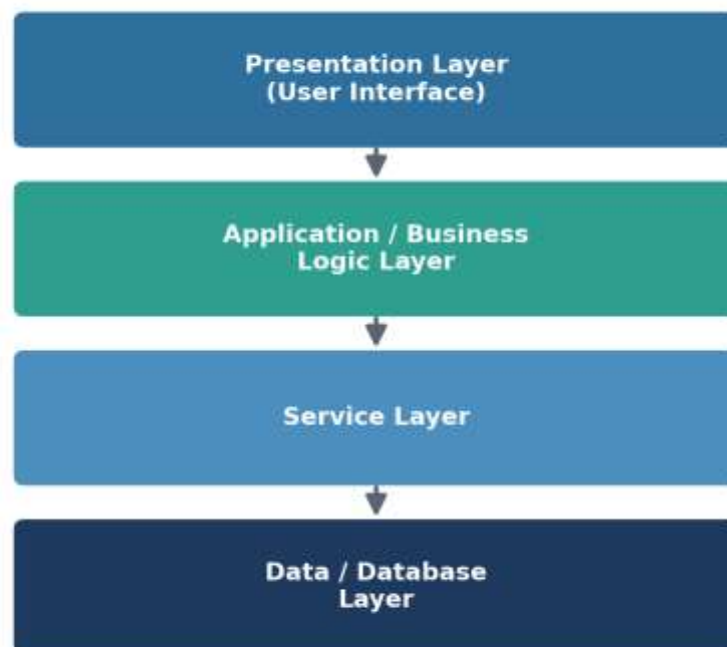


Figure 6.1 — A generic layered architecture: presentation, application logic, service, and data layers.

(c) Repository Architecture

- All system data is managed in a **central repository** accessible to all components; components interact only through the repository, not directly with one another.
- Example: an IDE, where the compiler, editor, and debugger all share a central repository of program data.

(d) Client–Server Architecture

- System functionality is organised into services, each delivered from a separate server; **clients** access these servers to use the services.
- Example: a multi-user, web-based film and photograph library system.

(e) Pipe and Filter Architecture

- Models the run-time organisation of a system where functional transformations (**filters**) process inputs and produce outputs.
- Data flows from one filter to the next through **pipes**, being transformed as it moves through the sequence.

3. Service-Oriented Architecture (SOA)

- **SOA** decomposes an application into loosely coupled, reusable services that expose functionality through standardised interfaces (e.g. web services).
- Services are designed to be independent, composable, and interoperable across different platforms and technologies.

Example — Choosing an architecture

- ▶ A blogging platform's request-handling logic fits MVC: Model = posts/comments data, View = rendered HTML page, Controller = request handler.
- ▶ An enterprise application split into UI, business-logic, and data-access tiers is a classic layered architecture.
- ▶ A compiler with separate lexer, parser, and code-generator stages, each transforming its input, is a pipe-and-filter design.

Quick Revision Points

- ✓ Software architecture defines a system's high-level components and their interactions.
- ✓ Key patterns: MVC, Layered, Repository, Client-Server, and Pipe-and-Filter — each suited to different problems.
- ✓ SOA builds systems from loosely coupled, standardised, reusable services.
- ✓ A clear architecture improves communication, supports analysis, and enables large-scale reuse.

Week 7: Software Implementation

Implementation converts a software design into working, executable programs. This week covers coding practice, debugging, reuse, version control, host-target development, IDEs, and build tools.

1. What is Software Implementation?

Software Implementation

The process of converting a software design into an executable program, including writing source code, developing modules, integrating components, debugging, and unit testing.

- **Implementation stage activities:** coding, unit testing, integration, debugging, and deployment preparation.

2. Coding Standards and Good Practice

- **Coding standards** are rules used when writing programs; they improve readability, reduce errors, simplify maintenance, and support teamwork.
- **Good programming practices:** meaningful variable/method names, clear comments, avoiding duplication, breaking programs into functions/methods, and handling errors properly.

3. Debugging

Debugging

The process of finding and fixing errors in a program.

- **Types of errors:** syntax errors, logical errors, and runtime errors.
- **Common debugging tools:** breakpoints, logs, and console output.

4. Software Reuse

- Using existing components instead of building everything from scratch — e.g. libraries, APIs, frameworks, and existing modules.
- **Advantages:** faster development, reduced cost, improved reliability, less coding effort, and better productivity.

5. Version Control Systems

- Track changes made to source code over time; popular tools include **Git, GitHub, GitLab, and Bitbucket**.

6. Host–Target Development

- **Host system** — the computer used to develop the software.
- **Target system** — the device on which the software actually executes.

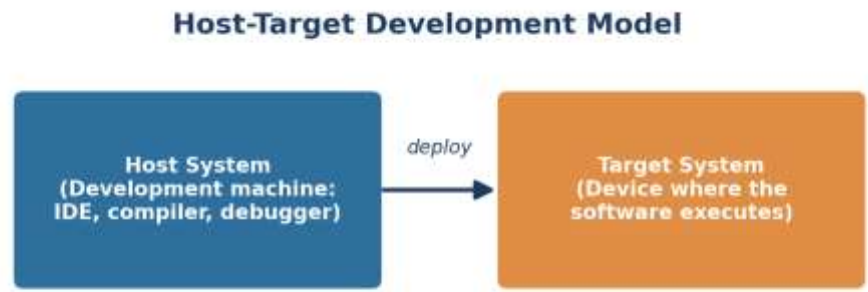


Figure 7.1 — Host-target development: software is built on the host and deployed to the target.

7. Integrated Development Environments (IDEs)

- An IDE bundles the tools needed for development: **code editor, compiler, debugger, emulator, and build tools**.
- Examples: **IntelliJ IDEA** (Java), **Android Studio** (Android), **PyCharm** (Python), **Visual Studio** (C#).

8. Build Tools and Deployment

- **Build tools** automate the software build process, e.g. **Maven, Gradle, Ant**.
- **Deployment considerations:** hardware requirements, software compatibility, performance, and communication between components.
- **Common implementation challenges:** managing large codebases, version conflicts, integration issues, security vulnerabilities, and tight deadlines.

Example — A typical implementation workflow

- ▶ A developer writes a new feature in IntelliJ IDEA (the host system's IDE) and commits the change to a Git branch.
- ▶ Automated unit tests run via the build tool (e.g. Gradle) to catch regressions immediately.
- ▶ Once merged, the build tool compiles and packages the app for the target system — for example, an Android phone.

Quick Revision Points

- ✓ Implementation turns a design into working code through coding, integration, debugging, and unit testing.
- ✓ Coding standards and good practice reduce errors and simplify long-term maintenance.
- ✓ Reuse (libraries, APIs, frameworks) speeds up development and improves reliability.
- ✓ Host-target development separates where software is built from where it finally runs.
- ✓ IDEs and build tools (Maven, Gradle, Ant) automate and support the development workflow.

Week 8: Software Testing

Testing checks whether software behaves as intended and helps discover defects before real use. This week covers verification vs validation, the stages of testing, test-case selection techniques, and test-driven development.

1. What is Software Testing?

Software Testing

The process of executing a program using test data and checking the results, in order to verify that the software behaves as intended and to discover defects before it is used.

- Important limitation: **testing can show the presence of errors, but it can never prove the complete absence of defects.**

2. Verification vs Validation

- **Verification** — “Are we building the product right?” Checks whether the software meets its specification.
- **Validation** — “Are we building the right product?” Checks whether the software meets customer expectations.

3. Stages of Software Testing

- **Development testing** — done during development by the development team to find bugs.
- **Release testing** — done by a separate testing team before release; focuses on validation and typically uses black-box testing.
- **User testing** — done by real users/customers in their own environment.

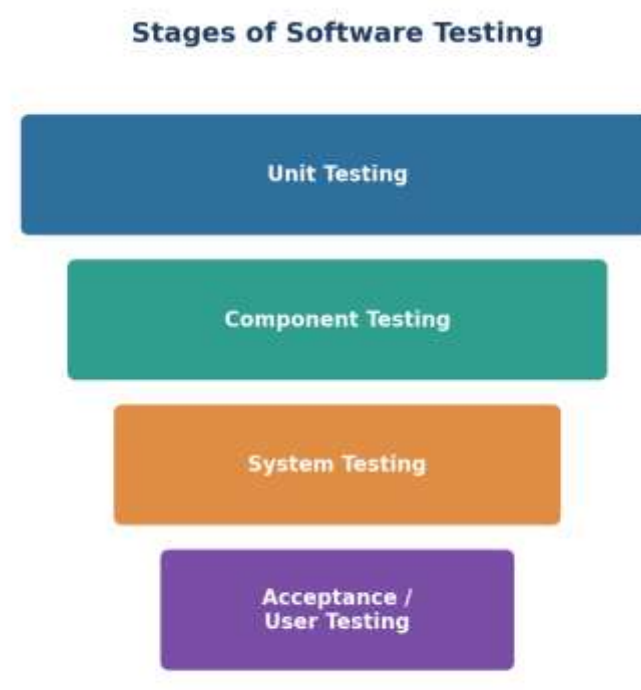


Figure 8.1 — Levels of software testing, from individual units up to full acceptance testing.

(a) Unit Testing

- Focuses on small parts of the program — a single method, class, or function.
- **Purpose:** check each method works correctly, test object-class behaviour, find defects early, support debugging, and reduce problems before component testing.

- Some methods cannot be tested alone and need a **sequence** of calls — e.g. to test ``shutdown()``, the system may first need to run ``restart()``.
- **Choosing test cases: Partition testing** (identify groups of inputs sharing common characteristics, then pick cases from each group) and **Boundary value testing** (test on, near, and away from partition boundaries).
- **Automated unit testing** uses frameworks such as **JUnit** (Java), **NUnit** (.NET), **PyTest** (Python), and **Jest** (JavaScript).

(b) Component Testing

- Focuses on interfaces between units, data passed between components, and interaction errors — e.g. testing a login form together with database validation.

(c) System Testing

- Checks the complete, integrated system: errors from component interaction, functional requirements, non-functional requirements, and overall behaviour.

(d) Release Testing

- Tests a system version before delivery to users, ideally by a separate testing team, focusing on whether the system is “good enough” for use.

(e) Performance Testing

- Checks whether the system can handle its expected workload; **stress testing** is a type of performance testing — e.g. testing below then well beyond an expected 300 transactions/second.

(f) User Testing

- **Alpha testing** — selected users test early versions with developers present.
- **Beta testing** — a larger group of users test the software before final release.
- **Acceptance testing** — the customer decides whether to accept or reject the system.

4. Test-Driven Development (TDD)

- Tests are written **before** the program code; code is then developed to pass these tests, and the tests are re-run automatically after every change.

Example — Testing a login feature

- ▶ Unit test: verify the ``validatePassword()`` method rejects passwords shorter than 8 characters.
- ▶ Partition testing: test valid usernames, empty usernames, and usernames with special characters as separate partitions.
- ▶ Boundary value testing: test password length exactly at 8 characters, at 7, and at 9.
- ▶ System test: verify a user can log in and reach their dashboard end-to-end.

Quick Revision Points

- ✓ Testing shows errors exist but can never prove a system is defect-free.
- ✓ Verification checks against the spec; validation checks against customer expectations.
- ✓ Testing progresses through unit → component → system → release/user testing.
- ✓ Partition and boundary-value testing are the two core techniques for selecting good unit test cases.
- ✓ TDD writes tests first, then code, running tests automatically after every change.

Week 9: Software Maintenance

Software maintenance covers all the modifications made after a software product is delivered. This week covers why maintenance is needed, its four types, its cost, and reverse/forward re-engineering of legacy systems.

1. Introduction

Software Maintenance

A widely accepted part of the software development lifecycle, covering all modifications and updates made to a software product after its initial delivery.

2. Why Maintenance is Necessary

- **Market conditions** — changing policies (e.g. taxation rules) can force software changes.
- **Client requirements** — customers may request new features or functions over time.
- **Host modifications** — changes to hardware or the host OS require adaptive software changes.
- **Organisational changes** — business-level changes such as downsizing or acquiring another company.

3. Types of Software Maintenance

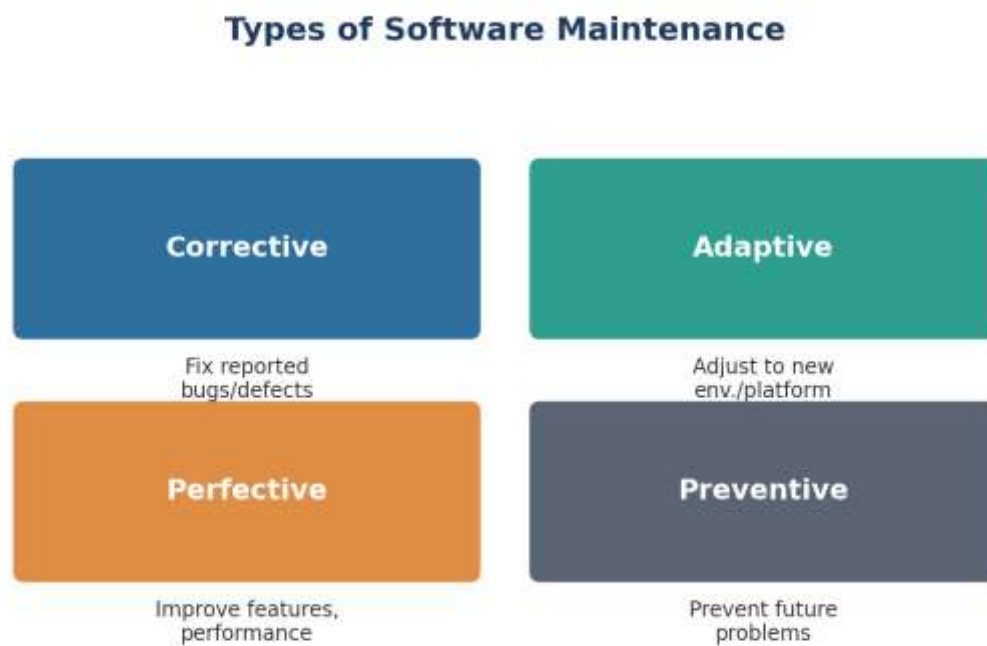


Figure 9.1 — The four recognised types of software maintenance.

- **Corrective maintenance** — modifications made to correct or fix problems discovered by users or reported via error reports.
- **Adaptive maintenance** — updates that keep the product current with a changing technology and business environment.

- **Perfective maintenance** — changes that keep the software usable long-term: new features, refined requirements, improved reliability and performance.
- **Preventive maintenance** — changes made to head off problems that are not yet significant but could become serious in future.

4. Problems With Software Maintenance

- Maintenance work is often more expensive and time-consuming than it should be.
- It is frequently carried out using **ad-hoc techniques** rather than a disciplined process.
- Most software needing maintenance consists of **legacy products**, which compounds these problems.

5. Estimating Maintenance Cost

- **Boehm (1981)** proposed a formula for estimating maintenance cost as part of the **COCOMO** cost-estimation model.
- Boehm's approach expresses maintenance cost in terms of the **Annual Change Traffic (ACT)** — the proportion of a system's source code that changes in a typical year.

6. Software Re-Engineering

- **Reverse engineering** — recovering the design and requirements specification of a product by analysing its existing code.
- **Forward engineering** — building the desired new software from the specifications recovered through reverse engineering.
- **Component reusability**: OOP objects tend to be narrowly specific and less reusable elsewhere, whereas modular programming produces modules that can be reused across many programs. **Component-Based Software Engineering (CBSE)** is a discipline built entirely around reusing such components.

7. Legacy Software

- A **legacy system** is best defined as any software system that is hard to maintain.
- Typical problems: poor documentation, unstructured (“spaghetti”) code, and a lack of personnel who understand the product — many legacy systems were built a long time ago.

Example — Maintenance in action

- ▶ Corrective: fixing a bug where invoices display the wrong currency symbol for EU customers.
- ▶ Adaptive: updating an app to work with a new version of Android after an OS update.
- ▶ Perfective: adding a dark-mode feature and speeding up report generation based on user requests.
- ▶ Preventive: refactoring a fragile module before it causes a serious failure under future load.

Quick Revision Points

- ✓ Maintenance covers all post-delivery changes: corrective, adaptive, perfective, and preventive.
- ✓ Maintenance is often more expensive than expected, especially for poorly documented legacy systems.
- ✓ Boehm's COCOMO model estimates maintenance cost using Annual Change Traffic (ACT).
- ✓ Reverse engineering recovers design/requirements from code; forward engineering builds new software from them.

Week 10: Software Project Management

Software project management plans, organises, monitors, and controls a software project — balancing budgets, schedules, resources, risk, and people.

1. What is Software Project Management?

Software Project Management

The process of planning, organising, monitoring, and controlling a software project so that it is delivered on time, within budget, and to the required quality.

- Projects are constrained by **budget, schedule, resources, and customer expectations**.

2. Why Software Projects Are Hard to Manage

- **Challenge 1: Software is intangible** — progress is harder to see or measure than in physical construction.
- **Challenge 2: Projects are unique** — each project differs, limiting how much past experience transfers directly.
- **Challenge 3: Software processes vary** — there is no single process that fits every project.

3. Role of the Software Project Manager

- **Project planning, risk management, people management, reporting, and proposal writing.**
- **Project planning** includes: defining objectives, deliverables & milestones, the Work Breakdown Structure (WBS), activity identification and sequencing, and scheduling.

4. Project Risk Management

Risk

An uncertain event that may negatively affect a project — e.g. a team member resigning, technology failure, requirement changes, or budget reduction.

- **Project risks** — e.g. staff shortages, delays in delivery.
- **Product risks** — e.g. poor performance, security vulnerabilities.
- **Business risks** — e.g. market changes, customer cancellation.



Figure 10.1 — The risk management process is a continuous cycle throughout the project.

- **Risk identification** — listing possible risks (staff turnover, requirement changes, hardware failures, security threats, supplier delays).
- **Risk analysis** — assessing the probability and impact of each identified risk.
- **Risk planning** — deciding how to respond to (avoid, reduce, or accept) each significant risk.
- **Risk monitoring** — tracking team morale, project progress, budget status, and technical issues throughout.

5. Managing People

- People are the most valuable asset in a software project: motivating staff, resolving conflicts, improving communication, and building strong teams matter as much as technical planning.
- A typical software team includes: **Project Manager, Business Analyst, Software Developers, Testers, UI/UX Designers, and DevOps Engineers.**
- **Characteristics of effective teams:** clear goals, open communication, trust, cooperation, strong leadership, and shared responsibility.

6. Project Scheduling

- **Main steps:** identify activities → estimate effort → determine dependencies → allocate resources → create schedule → monitor progress.
- A **project activity** is a task that must be completed (e.g. requirements analysis, system design, database design, coding, testing, deployment).
- A **milestone** is a significant project event or achievement (e.g. “Requirements Approved”, “Design Completed”, “Final System Released”).
- **Types of activity dependency:** Finish-to-Start, Start-to-Start, Finish-to-Finish.
- An **Activity Network Diagram** shows activities, their dependencies, and task sequence; a **Gantt Chart** displays the schedule visually over time.
- **Critical path** — the sequence of activities that determines the minimum overall completion time. It is the **longest path** through the schedule; no delay is allowed on it, since a delay to any critical activity delays the whole project.

Example — Risk management in practice

- ▶ Risk identified: 'Key backend developer may resign mid-project.'
- ▶ Risk analysis: high impact (delays delivery by weeks), medium probability.
- ▶ Risk planning: cross-train a second developer on the backend module now, before the risk materialises.
- ▶ Risk monitoring: track developer workload and morale weekly as an early-warning signal.

Quick Revision Points

- ✓ Project management = planning, organising, monitoring, and controlling a project.
- ✓ Software projects are hard to manage because software is intangible, each project is unique, and processes vary.
- ✓ Risks are classified as project, product, or business risks and managed through identify → analyse → plan → monitor.
- ✓ The critical path is the longest activity sequence in the schedule — delays here delay the whole project.

Week 11: Quality Management

Software Quality Management ensures a system meets customer requirements, organisational standards, and expectations around reliability and maintainability, using planning, assurance, control, standards, reviews, and measurement.

1. What is Software Quality Management (SQM)?

Software Quality Management

Activities that ensure software meets customer requirements, organisational standards, reliability expectations, and maintainability requirements.

2. Quality Management Activities



Figure 11.1 — Quality Planning sets goals; Quality Assurance builds them into the process; Quality Control checks the result.

- **Quality Planning** — defines the product description, quality goals, review procedures, testing strategy, and standards to follow.
- **Quality Assurance (QA)** — process-focused: making sure the development process itself is followed correctly to meet quality standards.
- **Quality Control (QC)** — product-focused: checking the finished product against the defined standards.

3. Software Standards

Software Standards

Documented best practices that support quality assurance.

- **Purpose:** improve consistency, improve communication, reduce errors, and support maintainability.
- **ISO 9001** — a widely used standard for organisational quality procedures, requiring compliance with defined processes and documentation of quality activities.

4. Reviews and Inspections

- **Software reviews** — team members examine documents or code, check them against quality standards, and record findings as defect reports.
- **Software inspection** — a peer-review process where developers systematically examine source code to find **logical errors, missing functionality, coding-standard violations, and design inconsistencies**.
- **Inspection checklists** guide reviewers to look for the same categories of issue consistently across reviews.

5. Quality Management in Agile Development

- Agile builds quality in through practices such as **refactoring, Test-Driven Development (TDD), continuous testing, and shared team responsibility for quality**.
- **Traditional approach** — quality is the job of a dedicated QA team.
- **Agile approach** — quality is built into everyday development activities; the whole team works together to improve it.

6. Software Measurement

- Involves collecting quantitative data about **product metrics** and **process metrics**.
- **Process metrics** examples: number of defects discovered, review effectiveness, and development effort expended.

Example — Quality management in a project

- ▶ Quality planning: the team decides code must pass 90% unit-test coverage before release.
- ▶ Quality assurance: a process is enforced where every pull request requires at least one code review.
- ▶ Quality control: the QA team runs a final regression test suite before sign-off.
- ▶ Inspection: a peer inspection catches a logic error in the discount-calculation module before release.

Quick Revision Points

- ✓ SQM ensures software meets customer needs, standards, reliability, and maintainability requirements.
- ✓ Quality Planning sets the goals; Quality Assurance builds a good process; Quality Control checks the product.
- ✓ Standards (e.g. ISO 9001) document best practice and support consistency and communication.
- ✓ Reviews and inspections are structured, peer-based ways to catch defects early.
- ✓ Agile teams build quality into everyday practice (TDD, refactoring, continuous testing) rather than relying only on a separate QA team.

Week 12: Configuration Management

Configuration Management (CM) is the discipline of managing change in software systems throughout their life — tracking versions, building systems, controlling changes, and preparing releases.

1. What is Configuration Management?

Configuration Management (CM)

The collection of policies, processes, and tools used to manage changing software systems.

2. Why is CM Needed?

- Bugs are fixed; new features are added; customer requirements change; operating systems and hardware change; and multiple developers often work on the same project simultaneously — all of this must be tracked and coordinated.

Example — CM in a real project

- ▶ Developers create new code; testers verify the feature; managers approve it.
- ▶ The new version is stored, and Version 2.35 is released.
- ▶ If bugs appear in production, developers can roll back safely to Version 2.34.

3. The Four Configuration Management Activities

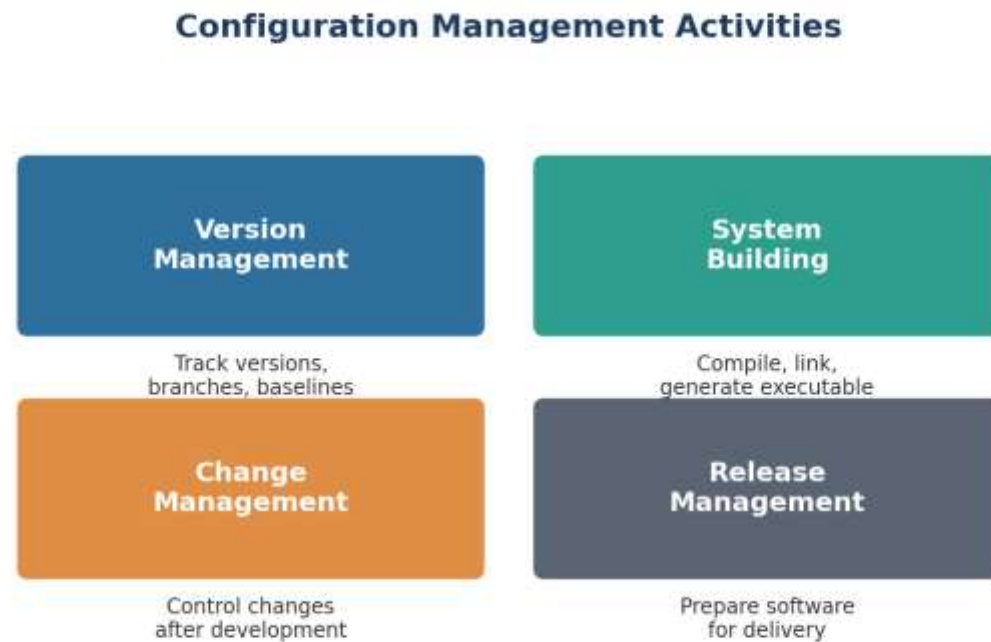


Figure 12.1 — The four core Configuration Management activities.

(a) Version Management

- Tracks every version of every software component so previous versions are preserved, developers can work safely, changes are never lost, and the full software history is maintained.
- **Key terms:** Configuration Item (SCI), Version, Baseline, Codeline, Branching, Merging, Release, Mainline.
- **Software Configuration Item (SCI)** — any item that must be managed during software development (code, documents, test data, etc.).
- **Baseline** — an approved version of the software that acts as a stable reference point.
- **Version control tools:** Git, GitHub, GitLab, Bitbucket.

(b) System Building

- Assembling components, compiling source code, linking libraries, and generating the final executable software.

(c) Change Management

- Controls software modifications after development; changes may originate from customers, testers, developers, or management.

(d) Release Management

- Prepares software for delivery to customers. A release typically bundles the **executable program, configuration files, data files, and documentation**.
- **Release types:** **Major release** (e.g. Version 2.0), **Minor release** (e.g. Version 2.1), **Patch** (e.g. 2.1.1), and **Hotfix** (an emergency fix).

Example — Git-based teamwork example

- ▶ Three developers each create a separate branch to work independently without interfering with each other.
- ▶ After testing, all branches are merged into the Main branch.
- ▶ In a university Student Management System: Developer A builds the Login module, Developer B builds the Attendance module; Git stores every change, Jenkins automatically builds the project, and after testing, Version 2.0 is released to students.

4. Advantages of Configuration Management

- Better teamwork, easy tracking of software versions, prevents data loss, faster recovery, easier maintenance, improved software quality, and more reliable releases.

5. DevOps

DevOps

A software development approach that brings Development (Dev) and Operations (Ops) teams together to build, test, release, and maintain software more quickly and reliably.

Quick Revision Points

- ✓ CM is the set of policies, processes, and tools for managing changing software systems.
- ✓ The four CM activities are Version Management, System Building, Change Management, and Release Management.
- ✓ A baseline is an approved, stable version of software; branching/merging let multiple developers work in parallel safely.
- ✓ Releases are classified as major, minor, patch, or hotfix.
- ✓ DevOps unifies development and operations to release software faster and more reliably.